

Your AI writes perfect SQL.

*It's still giving you
the wrong number.*

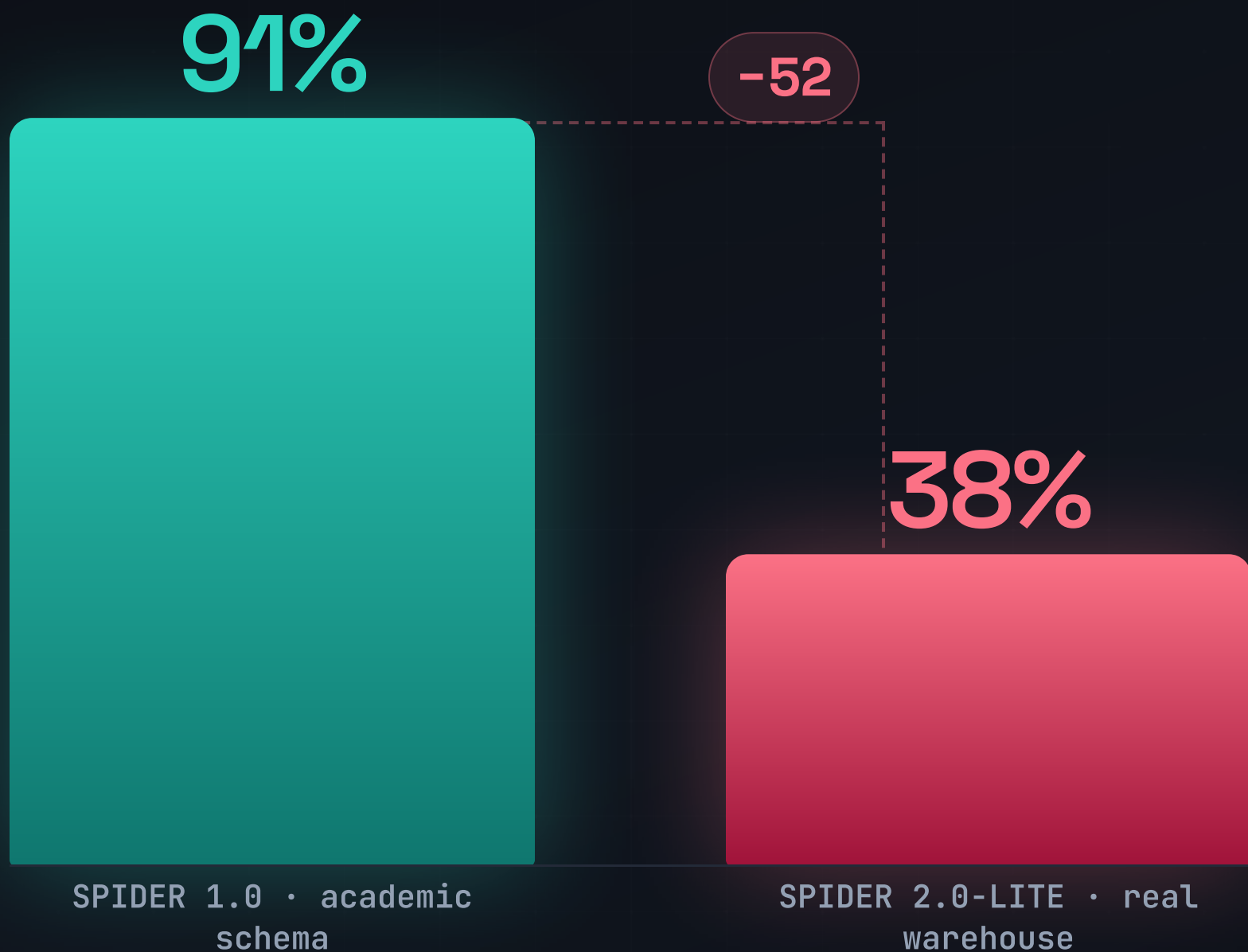
Why the failure isn't syntax — and why a
bigger model won't fix it.

● ● ● generated_query.sql · ran in 1.4s

```
SELECT region, SUM(amt_ttl_c) AS revenue
FROM   fct_orders o
JOIN   dim_geo g ON o.cust_id_alt = g.cust_id
WHERE  flg_2 = 'Y'
GROUP BY 1;
```

0 errors · wrong answer

The cliff is 50 points deep.



Agents solve ~91% of clean textbook schemas. On real warehouses, today's leaderboards sit at 38–59% depending on the split. Snow 59.05% • dbt 39.71% • Lite 37.84%

Nothing about the model changed between those two bars. **Only the database did.**

WHAT EVERYONE ASSUMES

“The model can’t

write good SQL.”

It writes **flawless** SQL.
That’s the problem.

Syntax is solved. Nothing in the query looks wrong —
which is exactly why nobody checks it.

Nobody documented any of this.

fct_orders_v3

amt_ttl_c	numeric
amt_ttl_g	numeric
flg_2	char(1)
dt_eff	date
dt_eff_utc	timestamp
cust_id_alt	varchar
cust_id_legacy	varchar
rgn_cd_x	varchar

+ 792 more columns

800

columns per database on average. Some over 1,000.

3

plausible join paths. One is correct.

0

of your business definitions live in the schema. They live in a Confluence page.

The model isn't guessing at syntax. It's guessing at **meaning** — from column names that carry none.

undocumented columns

ambiguous joins

dialect quirks

governance rules

All three run. One is right.

A orders → geo → SUM(amt_ttl_c) \$4.18M ?

B orders → items → geo → SUM(line_amt) \$4.61M ?

C orders → geo → SUM(amt_ttl_g) net of returns \$3.92M ✓

The model doesn't fail at *writing* SQL. It fails at knowing *which* SQL your business calls correct — and no error message marks the difference.

Pick the wrong one and revenue is off by **\$690,000** — with a query that looks textbook-clean.

It doesn't crash.
It **answers.**

REGION	REVENUE
North America	\$4,182,900
EMEA	\$2,940,118
APAC	\$1,708,442
LATAM	\$611,205

**PLAUSIBLE.
WRONG.**

A crash you notice in seconds. A confidently wrong number ships to a board deck and lives for a quarter.

Not a bigger model. A semantic layer.

dbt 11 questions · vendor-run

Raw text-to-SQL



90.0%

Semantic layer



98.2%

Cube 100 questions · 3 frontier models

Raw text-to-SQL



~49%

Semantic layer



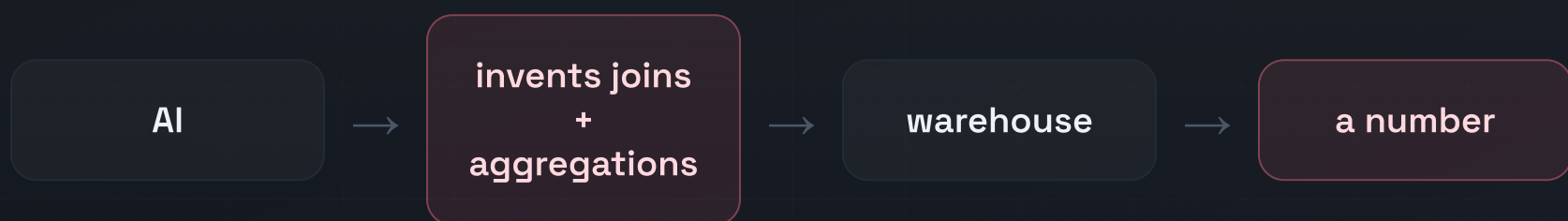
~68%

They disagree on the ceiling. They agree on the direction — and on this: **which model you use matters less than whether it has a semantic layer.**

The structure did the work — **not the model.**

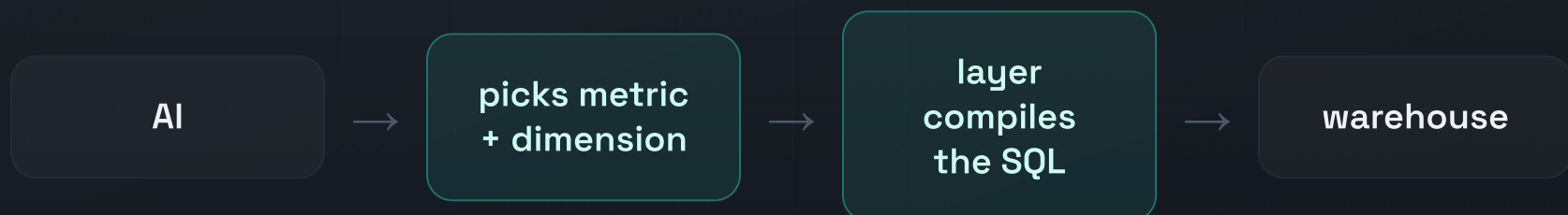
The error becomes impossible, not rarer.

✗ Text-to-SQL



Every run can pick a different wrong path.

✓ Semantic layer



The model never writes a join. Deterministic, every run.

GENERALIZES FAR BEYOND SQL

When AI keeps getting it **subtly wrong**, don't reach for a bigger model.

Narrow what the model is allowed to decide.

Define the metric once. Let the AI choose from it.

Text-to-SQL or semantic layer — which is your team running? 📝

#SQL · #DataEngineering · #SemanticLayer · #AIEngineering · #dbt